MPA (Multi-Proof Algorithm) Consensus Specification

1. Introduction and Background

The **Multi-Proof-Algorithm (MPA)** is a hybrid consensus mechanism implemented on top of the ADAM cooperative consensus framework. It combines Proof-of-Stake (PoS) with specialized cryptographic proof mechanisms to incentivize node operators, lock-up liquidity, and ensure secure block production.

MPA introduces four co-existing staking proof types:

1. **Proof of Stake (PoS - Baseline)**: Standard coin age-based block production.
2. **Proof of Lock (PoL)**: Staking using output scripts containing CLTV or CSV timelocks, awarding extra mining weight for longer lock durations.
3. **Proof of Burn (PoB)**: Burning coins to a designated unspendable address, providing a mining weight multiplier that decays linearly over time.
4. **Proof of Masternode (PoM)**: Allocating mining weight based on active Masternode collateral and its continuous uptime.

These weight metrics directly scale the difficulty targets during block validation, creating a multi-tiered consensus topology where resource commitment translates directly to consensus power.

---

2. Consensus Architecture & Parameters

MPA is activated at a specific block height `nPoMBLHeight`. Its core configuration resides in `src/consensus/params.h` and is defined per network in `src/chainparams.cpp`.

Core Parameters

- `nPoMBLHeight`: The block height at which MPA consensus rules activate. PoMBL stands for **Proof of Masternode, Burn and Lock**. Below this height, the network operates under legacy or version 11 ADAM rules.
- `nPoMBLTargetSpacing`: Spacing target for block production (configured to 30 seconds).
- `mBurnAddresses`: A map containing registered unspendable burn addresses and their active starting heights.

Network Activation Heights

Network	`nPoMBLHeight` (UPGRADE_POMBL)	Enforced Block Version
Mainnet	2000	Version 12
Testnet	400	Version 12
Regtest	300	Version 12

---

3. Mining Kernel Weight Calculations

The probability of mining the next block under the hybrid MPA system is governed by the kernel evaluation equation:

$$\text{KernelHash} < \text{Target} \times \text{Weight}_{\text{Total}}$$

For a given transaction output (UTXO) or Masternode collateral, its mining $\text{Weight}_{\text{Total}}$ is calculated using the following rules implemented in `CalculateMPAWeight()` in `src/kernel.cpp`:

3.1. Proof of Stake (PoS - Baseline)

$$W_{\text{PoS}} = \text{Amount}$$

3.2. Proof of Lock (PoL)

Applicable to transaction outputs locked using absolute (`OP_CHECKLOCKTIMEVERIFY`) or relative (`OP_CHECKSEQUENCEVERIFY`) timelocks of duration L blocks up to T_{MAX}:

$$W_{\text{PoL}} = \text{Amount} \times \left(1 + \gamma \cdot \min\left(\frac{L}{L_{\text{MAX}}}, 1.0\right)\right)$$

- γ is the lock multiplier parameter (default: `2.0`, giving up to a 3x weight bonus).
- L_{MAX} is the maximum lock duration evaluated (default: `50,000` blocks).

3.3. Proof of Burn (PoB)

Coins voluntarily sent in standard non-coinbase / non-coinstake transactions (`!tx.IsCoinBase() && !tx.IsCoinStake()`) to a registered unspendable burn address (`KTburnCYcDjF1GsXa5k3tc3eZrVVbqefyWr` / `KTburnUHdmKMVx4z8BcnZYZCWYKxRgfZp95` on Mainnet, `ktBurn42LtQP2pJ2fS5X2kpRx4Sd86kNgx4` / `ktBurnKmNRzVXVVVDXxFhhKytkH5x2yqsCG` on Testnet) receive a substantial personal PoB staking weight multiplier in `mapAddressBurns` that decays linearly to zero over time T (blocks elapsed since the burn block). Automatic protocol burns in `CoinBase` or `CoinStake` outputs (unclaimed Model D shares) permanently deduct from `nMoneySupply` as pure supply deflation and do **not** grant personal PoB staking weight:

$$W_{\text{PoB}} = \text{Amount} + \text{BurnAmount} \times \beta \times \left(1 - \frac{T}{T_{\text{MAX}}}\right)$$

- β is the burn incentive multiplier (default: `5.0`).
- T_{MAX} (`nBurnDecayBlocks`) is the decay threshold (`500,000` blocks on Mainnet, `10,000` blocks on Testnet/Regtest). Once $T \geq T_{\text{MAX}}$, the burn bonus decays to 0 (leaving the base staking UTXO amount).

3.4. Proof of Masternode (PoM)

Active Masternodes with unspent 4,200 KRISTA collateral C and active lifetime t_{active} (evaluated deterministically on-chain via `mapMasternodeCollateralOutpoints` and `IsMasternodeActiveAtHeight(..., 2880)` using the earliest active ping height in `mapMasternodeActiveHeights` or collateral coin height, with `mnodeman` as a secondary fallback):

$$W_{\text{PoM}} = C \times \left(1 + \alpha \cdot \min\left(\frac{t_{\text{active}}}{T_{\text{MAX}}}, 1.0\right)\right)$$

- α is the masternode lifetime multiplier (default: `1.0`, yielding up to 2x weight).
- T_{MAX} (`nMasternodeUptimeLimit`) is the maximum lifetime maturity (`100,000` blocks on Mainnet, `10,000` blocks on Testnet/Regtest).
- If a Masternode fails to broadcast an on-chain heartbeat ping within the 2,880-block activity window (or falls out of `ENABLED` status), its PoM bonus becomes inactive (`t_active = 0`).

---

4. Long-Living Masternode Quorums (LLMQs)

To support secure leader election and signature aggregation, LLMQs are implemented using a **Hybrid BLS12-381 + secp256k1** cryptographic architecture (`CBLSSignedData` and `CQuorumSignature::AggregateBLSSignatures`). Each quorum member signs the block hash (`blockHash`) using its deterministic **BLS12-381** (`blst`) key (`DeriveBLSFromCKey`), authorized by its permanent `secp256k1` Masternode identity key and persistently pinned in LevelDB (`PinBLSPubKey`). During verification (`CQuorumSignature::Verify`), after verifying member identities and pinned BLS public keys, the signing members' BLS12-381 public keys (`CBLSPubKey::Aggregate`) and signatures (`CBLSSignature::Aggregate`) are aggregated and verified via a single pairing equation.

Quorum Election & Size

- **Quorum Size**: Exactly **5 members**.
- **DKG Interval**: DKG sessions run every **100 blocks** on Mainnet (until block 1,400) and every **10 blocks** on Testnet (until block 1,000) / Regtest (until block 800); after these heights, a new quorum is elected **every block (`nInterval = 1`)**.
- **Active Masternode Filtering**: No hardcoded key ID filtering or local wallet restrictions are applied on any network (Mainnet, Testnet, or Regtest). The election dynamically draws from all active and enabled Masternodes on the network.
- **Deterministic Fallback**: If fewer than 5 active masternodes are available, the DKG session manager falls back to electing quorum members from the registered miner pool (`GetAdamMinerPool(nHeight - 1)`). On Regtest, the registered miner pool is pre-populated with **15 deterministic keys** (derived from the index 0 to 14) to facilitate local testing, while on other networks it consists of registered miners (via Coin-Lock or PoW-Lock) and bootstrap miners.

Signature and Threshold Validation

- Quorum members sign the block hash using their deterministic **BLS12-381** (`DeriveBLSFromCKey`) keys authorized by their `secp256k1` identity keys, and signatures are aggregated via `CQuorumSignature::AggregateBLSSignatures`.
- The quorum signature `vQuorumSig` is populated in block headers once version is `>= 12` (when Standard Mode is active via height-gated `Consensus::UPGRADE_POMBL` at `nHeight >= nPoMBLHeight`: block 2,000 on Mainnet, 400 on Testnet, and 300 on Regtest). Note that Model D (`UPGRADE_MODELD`) is a separate upgrade height that activates at height 2,200 on Mainnet, 500 on Testnet, and 200 on Regtest.
- **Quorum Validation Threshold (`CQuorumSignature::Verify` in `src/llmq.cpp`)**:
  - **Prior to Model D Activation (`!NetworkUpgradeActive(nHeight, UPGRADE_MODELD)` on Mainnet / Testnet)**: The verification threshold is **0 signatures** (verification is bypassed to allow bootstrapping before active LLMQ quorums are required).

- **Once Model D is Active (Mainnet, Testnet, and Regtest)**: Both block generation (`miner.cpp`) and consensus verification (`llmq.cpp`) enforce a strict BFT majority threshold of `quorum.members.size() / 2 + 1` (i.e., **3 out of 5 signatures** for a standard 5-member quorum).

> **NOTE**
>
> **Quorum Threshold Distinction:** Do not confuse the **ADAM Cooperative Mining Threshold** (7 out of 11 rule for puzzle solutions `vAdamSolutions`) with the **LLMQ Signature Verification Threshold** (3 out of 5 rule on Mainnet for block signature `vQuorumSig`):
>
> - **ADAM Threshold**: Enforces that at least 7 out of 11 elected miners (`nAdamThreshold = 7` on Mainnet/Regtest, `3` on Testnet) must solve and submit their PoW puzzle solutions (`vAdamSolutions`) for the block to be accepted. This is part of the ADAM cooperative puzzle validation.
> - **LLMQ Threshold**: Enforces that at least 3 out of 5 LLMQ members must sign the proposed block hash using their hybrid BLS12-381 + secp256k1 keys (`vQuorumSig`). This is part of the decentralized block validation.

---

5. Consensus Enforcements & Validation Rules

When a block is received, the validation rules in `CheckBlock()` in `src/main.cpp` enforce the following checks when height ≥nPoMBLHeight :

1. **Version Enforcement**: The block version must be at least `12` . Block headers of version < 12 are strictly rejected with a `bad-version` code.
2. **Burn Output Integrity**: All transaction inputs spending from a registered burn address are strictly rejected at the mempool layer (`AcceptToMemoryPool`) and block connection layer (`ConnectBlock`).
3. **Locktime Script Enforcement**: Locked transaction outputs used for PoL must strictly conform to locked scripts and cannot be spent until their specified lock height/timestamp has passed.
4. **Quorum Vote Validation**: The quorum signature (`vQuorumSig`) included in the block header is validated against the active LLMQ public key.

---

6. Testing and Verification

The MPA consensus changes are fully covered by a dual test suite:

6.1. Unit Tests (`test_kristatech`)

The C++ unit tests in `src/test/mpa_tests.cpp` verify:

- Weight calculations for standard PoS.
- Correct decay parameters and decay calculations for Proof of Burn (PoB).
- Height and duration calculations for Proof of Lock (PoL) transaction inputs.

6.2. Functional Tests (`consensus_pombl.py`)

The Python functional test suite `test/functional/consensus_pombl.py` verifies the rules on a local Regtest network:

- Simulates block generation up to the activation height.
- Confirms block version < 12 rejection at the activation boundary.
- Confirms block version 12 acceptance.
- Verifies that sending funds to a burn address is accepted.
- Asserts that attempts to spend from a burn address are blocked and rejected with `bad-txns-invalid-outputs`.